

Software Trustworthiness Assessment via Large Language Models (LLMs)

Saeed Javani Jananloo

University of Coimbra, CISUC/LASI – Centre for Informatics and Systems of the University of Coimbra

Department of Informatics Engineering

Coimbra, Portugal

jananloo@dei.uc.pt

Abstract—Nowadays, almost everything in our daily lives is handled by software. From transport systems and autonomous systems to healthcare and financial contracts. So, trustworthiness has become critical. Software trustworthiness refers to the degree to which a software system can be trusted, in the aspects of reliability and security. It means the ability to behave as expected even when facing faults, attacks, or unexpected conditions. Static Code Analysis (SCA) tools can be used to identify security issues, such as software vulnerabilities. Though, SCA tools have a great number of False Positives (FPs) and False Negatives (FNs). Consequently, developers do not use them or spend a lot of time analyzing the alerts reported by such tools without knowing that all vulnerabilities are identified. On the other hand, Large Language Models (LLMs) are being used in software engineering tasks, such as code generation, code review, code documentation, and even bug detection. These capabilities have attracted the interest of researchers to their potential for automating Software Trustworthiness Assessment (STA). As the performance of LLMs in STA is not satiable yet, there is an emerging need to obtain the best results out of their great potential. Although there are many works about the use of LLMs for the Software Vulnerability Detection (SVD), to the best of our knowledge, no other works have evaluated their ability to perform STA. This Ph.D. has 4 goals: 1. To evaluate the ability of LLMs to do the STA, 2. To develop a Software Characterization Mechanism based on LLMs, 3. To develop the Software Compliance and Risk Assessment Process supported by LLMs, and 4. To create an LLM-based tool for STA.

Index Terms—Large Language Models, Software Security, Software Trustworthiness Assessment, Static Code Analysis

I. INTRODUCTION

A software vulnerability is a flaw or a weakness that can be exploited by a malicious actor to cause a failure of the confidentiality, the availability or the integrity of the system [1]. Thus, software developer teams must assess the source code from a security perspective before its release. Software Trustworthiness Assessment (STA) is the process of figuring out how dependable a system is, especially when it comes to security and reliability. The goal of STA is to make sure the software meets key standards and works properly, even when things do not go as planned. Therefore, it is mandatory for every software developer team, to detect and handle security issues in the early stages of the Software Development Life Cycle (SDLC). SCA is one of the most used vulnerability detection techniques because it provides a scalable way for security code review that can be used early in the life cycle,

does not require the system to be executable, and can be used on parts of the overall code base separately [2]. However, the problem is that many SCA tools have a recall rate near to 50%, thus they exhibit a performance not better than random guessing [2]. These results suggest that we cannot rely only on SCA because doing so would leave a large number of security issues undiscovered, let alone the large number of FPs that should be reviewed manually [2]. So, the result of the SCA tools needs to be reassessed cautiously by the human security experts, to increase the reliability of the process. The problem becomes worse when considering a large project with many files.

LLMs such as CodeBERT [3], CodeT5 [4], and DeepSeek [5] have proved themselves wonderfully in understanding programming languages. The ability of LLMs in code generation, code documentation, and test generation has encouraged many software developers to use them on a daily basis. These abilities motivated researchers to evaluate the feasibility of using LLMs in automating SVD. Researchers try to benefit from vast software engineering knowledge of these models in STA tasks.

In general, the goal of this Ph.D. project is to automate the STA process using the LLMs. In other words, we will investigate ways to replace the manual work done by security experts with LLMs. As most SCA techniques have high rates of FPs or FNs currently, our goal is to improve both or, at least, reduce one of them while not increasing the other. If we want to break this process into more detailed categories, we can enumerate them as: 1. we will evaluate the ability of LLMs to do the STA, 2. we want to develop a software characterization mechanism based on LLMs, 3. we will develop the software compliance and risk assessment process supported by LLMs, and 4. we will create a tool for STAs based on LLMs.

II. RELATED WORK

Liu *et al.* present the idea of combining code-specific information, including Data Flow Graphs (DFG) and Abstract Syntax Trees (AST), with the Chain of Thought (CoT) prompts to get better results from the LLMs [6]. The datasets that they tested are the Devign dataset [7] and the Reveal dataset [8]. Their focus is on Accuracy measurement for the validation of the results, and their tested LLM is OpenAI's GPT-3.5-turbo. Their experiments show that role-based

prompts and CoT have increased the initial accuracy of the basic prompt. But they could not get a better result than 55%.

Zhang *et al.* propose multi-step prompts to get a better answer from the OpenAI's ChatGPT-4. Since ChatGPT-4 can memorize multi-step interactions, they try CoT prompting for SVD [9]. They first ask the ChatGPT to describe the intent of the given code, and then they continue to ask ChatGPT about the vulnerability of the given function. Based on the results they published, this CoT prompting led them to reach an Accuracy of 0.741 for C/C++ functions.

Zhou *et al.* have tested both ChatGPT-3.5 and ChatGPT-4 with some types of prompts and reported the results [10]. The prompts they proposed, are: 1. simple prompt with just a task description, 2. prompt with a role description, 3. prompt including information about the project, 4. prompt including dangerous CWEs. As their results show, the best result happens when they give the prompt including dangerous CWEs to ChatGPT-4, with the F1-Score of 76.4, against 51.5 that was for CodeBERT.

Jiang *et al.* extract DFGs from the input code, then set the variables as nodes, and the dependencies between variables as directed edges [11]. They then encode the DFG into a graph representation using a Graph Neural Network (GNN) and combine this graph representation with LLM—the GNNs aggregate features from connected nodes to create embeddings for the graph. The DFG embeddings are concatenated with the code embeddings generated by pre-trained models (CodeT5, CodeBERT). These combined features are input into a classifier for vulnerability detection. Their method achieves an accuracy of 65% on the Devign dataset and an F1-Score of 48% on the Reveal dataset.

Right now, using LLMs in SVD faces 3 big voids that should be filled:

- **Small Input Granularity:** Currently, LLM-based SVD solutions mostly target the function level. Therefore, these approaches may not perform well when facing a wider range of programs, such as classes or a whole project. Moreover, they could overlook vulnerabilities that span multiple functions or classes[12]. That is mainly because of the limited context length that most LLMs can process.
- **Lack of High-quality Vulnerability Dataset:** Existing vulnerability datasets often contain notable noise, ranging from 40% to 75% [13]. This issue stems from a mistake in their presumption: "Every function that has been modified during the Vulnerability-Fixing Commits (VFCs), is vulnerable". But it is obvious that not all those changes pertain to security threats [13]. Some of the modifications might be because of other reasons, such as code refactoring, performance-related issues, or even adding new features.
- **Less-frequent vulnerabilities get overlooked:** As LLMs are pre-trained models, they are trained based on the most

frequently occurring phenomena. So, this is inevitable for them to have little knowledge about the least-frequent ones, or at least, ignore them because of their rareness. This leads them to have a low detection rate for rare vulnerability types.

III. GOAL AND APPROACH

This section describes the timeline for this Ph.D. project, the preliminary results, and the potential threats to validity.

A. Timeline

The proposed plan started in December 2024 and is planned to be completed by September 2029. In general, we want to perform the STA process via LLMs. Our focus is on SCA, not running the software and doing the intrusion-injection. So, we want to use LLMs to decide on the software security characteristics, based on its code, without running. In this direction, the main activities are described below:

- 1) As shown in Fig. 1, we want to evaluate the ability of LLMs to do the STA. To do this, we will evaluate the response of LLMs against the human security experts in the comparison stage of Security Characterization of Open-Source Functions using Logic Scoring of Preference (SCOLP) [14]. This characterization is based on Logic Scoring of Preferences (LSP) [15]. In LSP, the automatic process results were validated by human experts; But now, we will evaluate the possibility of validating these results by LLMs. As we know, there is a huge difference between the output of an LLM when facing a simple prompt and a well-crafted one. Also, as LLMs are good few-shot learners [12], In-Context Learning (ICL) and Few-Shot prompting will be used in this part to designate the best prompts possible.
- 2) We want to develop a software characterization mechanism based on LLMs. In the previous step, we wanted to evaluate the ability of LLMs against the

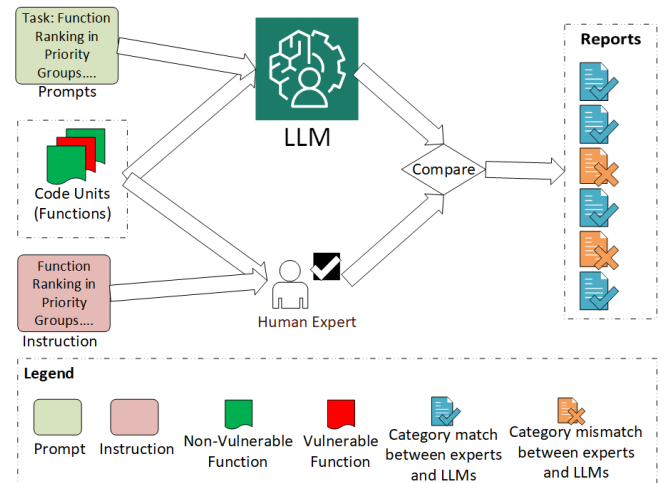


Fig. 1. The preliminary experiment

human security experts, but now, we will develop the whole Software Characterization Mechanism based on the LLM. All of the sub-operations of the Software Characterization process will be done by LLMs. In other words, LLMs will be used to complement our information about the source code to guide SCA and thus improve security assessment.

- 3) We will develop the software compliance and risk assessment process supported by LLMs. Software risk assessment is the process of identifying and analyzing potential risks in software development and deployment. Also software compliance assessment is the process of evaluating whether a software system adheres to regulatory standards and organizational policies, to ensure that it meets legal, security, privacy, and quality requirements. This LLM-based process will be designed to evaluate potential risks, such as vulnerabilities or scalability issues, and also to check compliance with regulatory requirements. These evaluations could lead to an informed decision-making process. For example, this can shed light on areas that need additional security observations or even architectural adjustments to increase resilience.
- 4) To develop the Software Compliance and Risk Assessment tool. In this phase, we will create the compliance and risk assessment tool, based on the process we created in the previous phase.

B. Potential Threats

Two potential threats to this Ph.D. were identified. First, difficult to run and obtain Static Analysis Tools (SATs) as a baseline: to mitigate this threat, a thorough review of the leading open-source SATs, such as CodeBERT, AIBugHunter, and VulDeePecker was already performed, and were evaluated. Second, is running the State-of-the-Art LLMs locally on a single machine to perform the comparison between the results. As Meta's Ollama [16] tool, is an open-source project designed to help users run and experiment with LLMs locally.

The evaluation process for our proposed methods is to compare the results not only with the traditional SATs, like FlawFinder [17] and CppCheck [18], but also AIBugHunter [19], and VulDeePecker [20] as baselines to get a complete comparison.

IV. PRELIMINARY RESULTS

Here, we did a primary experiment for using LLM in STA. Fig. 2 shows the experiment setup for our methodology. In the first step, we chose some LLMs to perform the experiment on them. The LLMs are selected from a list of LLMs that

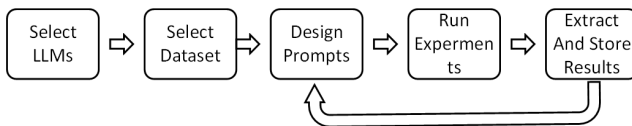


Fig. 2. The preliminary experiment methodology

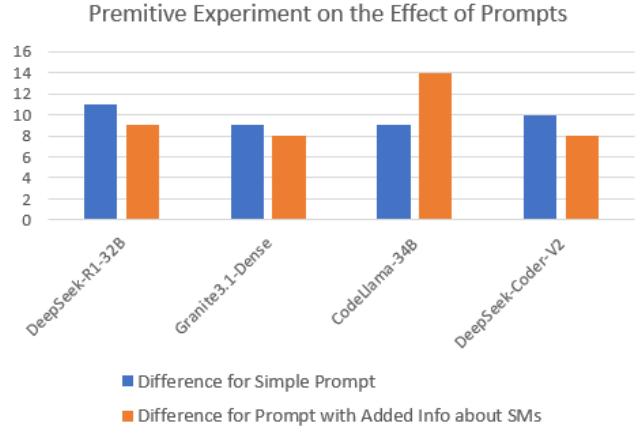


Fig. 3. The preliminary experiment results

could be run locally, and there are not any special criteria for their selection. Then we selected 10 functions from the Linux Kernel repository. Then designed and ran the prompts. Afterward, we gathered the results and compared them to the ground truth, which is the judgment of the security experts. For the quantization process, we first quantified the labels: 1 for Highest, 2 for High, 3 for Medium, 4 for Low, and 5 for Lowest. Then we defined the Distance measure. That is the absolute difference between the judgment of the LLMs with the experts. For example, if the function is High in the judgment of the experts, and the LLM says it is Low, then the difference for this experiment is 2. As we can see in Fig. 3 The results show that adding a piece of short information about the Software Metrics (SMs) to the initial prompt could lead to an increased rate of correct answers in 3 out of 4 LLMs. Although, this result is raw and we need to design more prompts and experiments to get more confident results. As we can see in the Fig. 3, the best result (lowest difference) came from the zero-shot prompt with added info about the SMs for two of the models: IBM's Granite3.1-Dense [21] and DeepSeekAI's DeepSeek-Coder-v2 [5], with 8 and 16 billions of parameter size respectively. The detailed results of the experiment for DeepSeek-Coder-v2 are illustrated in Table I.

Function	LLM Judgment	Expert Judgment	Difference
A	Low	Highest	3
B	Medium	Medium	0
C	Low	High	2
D	High	High	0
E	High	High	0
F	High	Highest	1
G	High	Medium	1
H	Low	Low	0
I	High	Medium	1
J	Low	Low	0
SUM			8

TABLE I
DEEPSEEK-CODER-V2 WITH ZERO-SHOT PROMPTING

V. CONCLUSION

Software trustworthiness assessment remains a critical challenge, and that makes enterprises invest in security measures continuously. This Ph.D. project focuses on using the strengths of LLMs to enhance the STA and assist software development teams in prioritizing their efforts to make their products safer. Preliminary findings have been shared, providing validation for the project's direction. Upcoming tasks include: (1) to evaluate the ability of LLMs to do the STA, (2) to develop a software characterization mechanism based on LLMs, (3) to develop the software compliance and risk assessment process supported by LLMs, and (4) to create a tool for STA based on LLMs.

ACKNOWLEDGMENT

This work is partially financed through national funds by FCT - Fundação para a Ciência e a Tecnologia, I.P., in the framework of the Project UIDB/00326/2025 and UIDP/00326/2025. Finally, the author thanks the support of the advisors Jose D'Abruzzo Pereira and Marco Vieira from the University of Coimbra.

REFERENCES

- [1] M. R. Shahid and H. Debar, "Cvss-bert: Explainable natural language processing to determine the severity of a computer security vulnerability from its description," in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2021, pp. 1600–1607.
- [2] K. Goseva-Popstojanova and A. Perhinschi, "On the capability of static code analysis to detect security vulnerabilities," *Information and Software Technology*, vol. 68, pp. 18–33, 2015.
- [3] Z. Feng, D. Guo, D. Tang, *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.
- [4] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.
- [5] D. Guo, Q. Zhu, D. Yang, *et al.*, "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [6] Z. Liu, Z. Yang, and Q. Liao, "Exploration on prompting llm with code-specific information for vulnerability detection," in *2024 IEEE International Conference on Software Services Engineering (SSE)*, IEEE, 2024, pp. 273–281.
- [7] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [8] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [9] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using chatgpt," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 276–277.
- [10] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024, pp. 47–51.
- [11] Z. Jiang, W. Sun, X. Gu, *et al.*, "Dfdept: Data flow embedding for enhancing pre-trained model based vulnerability detection," in *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, 2024, pp. 95–104.
- [12] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Transactions on Software Engineering and Methodology*, 2024.
- [13] Y. Li, T. Zhang, R. Widyasari, *et al.*, "Cleanvul: Automatic function-level vulnerability detection in code commits using llm heuristics," *arXiv preprint arXiv:2411.17274*, 2024.
- [14] J. D. Pereira and M. Vieira, "An approach to characterize the security of open-source functions using lsp," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, IEEE, 2023, pp. 137–147.
- [15] J. Dujmovic, *Soft computing evaluation logic: The LSP decision method and its applications*. John Wiley & Sons, 2018.
- [16] J. Morgan, *Ollama - get up and running with large language model*, 2023. [Online]. Available: <https://ollama.com/>.
- [17] D. A. Wheeler, *Flawfinder - a simple program that examines c/c++ source code and reports possible security weaknesses*, 2001. [Online]. Available: <https://dwheeler.com/flawfinder>.
- [18] D. Marjamäki, *Cppcheck - a tool for static c/c++ code analysis*, 2007. [Online]. Available: <http://cppcheck.sourceforge.net>.
- [19] M. Fu, C. Tantithamthavorn, T. Le, *et al.*, "Aibughunter: A practical tool for predicting, classifying and repairing software vulnerabilities," *Empirical Software Engineering*, vol. 29, no. 1, p. 4, 2024.
- [20] Z. Li, D. Zou, S. Xu, *et al.*, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [21] A. M. Soria, *Granite 3.1 language models*, 2023. [Online]. Available: <https://huggingface.co/ibm-granite>.